

HotStuff: simulator BFT konsenzusa za poravnanje između banaka bez centralnog posrednika

Anja Vranješević

Primenjeni algoritmi u upravljačkim sistemima

Sažetak

U radu je opisana implementacija simulatora HotStuff protokola za vizantijski otporan konsenzus (Yin, Malkhi, Reiter, Golan Gueta, Abraham; PODC '19). Protokol je primenjen na konkretan problem — međubankarski poravnalni registar u kome četiri banke koje međusobno ne veruju jedna drugoj vode zajednički, potpuno uređeni dnevnik transakcija bez centralnog operatera. Implementirane su obe varijante iz rada: bazični HotStuff (Algoritam 2) i ulančani (chained) HotStuff (Algoritam 3). Simulator koristi konkurentno izvršavanje četiri čvora kao nezavisnih `asyncio` zadataka, komunikaciju porukama preko simulirane mreže i kriptografske potpise, a kroz pet scenarija demonstrira i normalan rad i ponašanje u prisustvu grešaka i vizantijskih čvorova.

1 Uvod i motivacija

Problem konsenzusa je jedan od centralnih problema distribuiranih sistema: kako više čvorova, koji rade nezavisno i komuniciraju nepouzdanom mrežom, mogu da se slože oko jedne zajedničke vrednosti ili oko istog redosleda niza operacija. U slučaju replikovane mašine stanja to znači da svaka ispravna replika izvršava iste komande istim redosledom, pa samim tim dolazi i do istog stanja.

Klasične varijante problema pretpostavljaju da čvorovi mogu samo da otkazu (crash), tj. da prestanu da rade. Vizantijski otporan konsenzus (BFT) rešava teži slučaj: neki čvorovi mogu da se ponašaju proizvoljno zlonamerno — da šalju protivrečne poruke različitim primaocima, da lažu o svom stanju ili da ćute kada bi trebalo da govore. Da bi sistem sa n čvorova tolerisao f vizantijskih, potrebno je $n \geq 3f + 1$.

Kao motivacioni primer u ovom radu koristi se poravnanje između banaka. Zamislimo četiri banke koje žele da vode zajednički registar transfera, ali nijedna ne želi da veruje centralnoj instituciji koja bi taj registar održavala. Bez zajedničkog uređenja transakcija javlja se problem *dvostruke potrošnje*: ista banka može istovremeno da pošalje dva transfera koja zbir svojih iznosa prevazilaze njeno stanje. Ako se banke ne slože oko toga koji transfer je prvi, može se desiti da jedni čvorovi prihvate jedan, a drugi drugi transfer, čime bi se registar razišao. HotStuff rešava upravo taj problem: obezbeđuje da sve ispravne replike vide isti redosled komandi.

Ključna ideja je da konsenzus određuje redosled komandi, a mašina stanja sudi o njima. HotStuff odlučuje samo o tome *kojim redom* se transferi primenjuju. Da li će pojedinačni transfer uspeti ili biti odbijen jeste deterministička funkcija tog redosleda, koja se identično izračunava na svakoj replici. Zbog toga se dve ispravne replike nikada ne mogu razići — čak ni oko odbijenog transfera — jer sude o istoj komandi nad istim prethodnim stanjem.

2 HotStuff algoritam

HotStuff je BFT protokol zasnovan na lideru, kod koga se u svakom pogledu (*view*) bira jedan lider koji predlaže sledeći blok komandi. Njegova glavna prednost u odnosu na ranije

protokole (npr. PBFT) jeste *linearna* složenost komunikacije i jednostavna promena lidera: ne postoji poseban potprotokol za promenu lidera, već je regularan predlog novog lidera ujedno i mehanizam prelaska na novi pogled.

2.1 Model sistema

U implementaciji je usvojena kanonska najmanja postavka iz rada: $n = 4$ čvora, od kojih najviše $f = 1$ može biti vizantijskih, a za kvorum je potrebno $n - f = 3$ glasa. Ove vrednosti su fiksirane radi jednostavnosti koda. Svaki čvor poseduje par Ed25519 ključeva i može da potpisuje i proverava poruke.

2.2 Blok stablo i kvorum sertifikati

Komande se ne prihvataju pojedinačno, već se pakuju u *blokove* (u kodu tipa `Node`) koji formiraju stablo. Svaki blok sadrži heš roditelja, komandu i broj pogleda u kome je predložen, a njegov identitet je SHA-256 heš tog sadržaja. Time se roditeljskim vezama gradi lanac unazad sve do korenskog, tzv. *genesis* bloka koji je isti na svim replikama.

Nad blokovima se definišu dve pomoćne relacije. Blok a *proširuje* (**extends**) blok b ako se b nalazi na putu roditeljskih veza od a ka korenu. Dva bloka su u *konfliktu* (**conflicts**) ako nijedan ne proširuje drugi, tj. leže na različitim granama stabla. Suština bezbednosti protokola je da dve ispravne replike nikada ne *izvrše* (**commit**) blokove koji su u konfliktu.

Kvorum sertifikat (**QC**) je dokaz da je kvorum čvorova glasao za istu trojku $\langle \text{tip, pogled, blok} \rangle$. U originalnom radu QC je jedan agregirani *threshold* potpis, dok se u ovoj implementaciji čuva eksplicitna lista od tačno tri različita Ed25519 potpisa. Bezbednosno su ekvivalentni — QC i dalje dokazuje da je kvorum potpisao istu trojku — razlika je samo u veličini autentifikatora ($O(n)$ umesto $O(1)$ bajtova).

2.3 Pravilo bezbednog glasanja (safeNode)

Bezbednost protokola počiva na tome kada je replici dozvoljeno da glasa za predloženi blok. Svaka replika pamti *zaključani* sertifikat (**lockedQC**) — QC bloka na koji se poslednji put „zaključala”. Predikat **safeNode** dozvoljava glas ako je zadovoljen bar jedan od dva uslova:

1. **bezbednosni uslov:** predloženi blok proširuje blok na koji smo zaključani; ili
2. **uslov živosti:** sertifikat kojim se predlog opravdava potiče iz višeg view-a od našeg zaključavanja.

Prvi uslov sprečava da se glasa za konfliktnu granu i time garantuje bezbednost. Drugi uslov omogućava da nas nov, ispravan lider, koji je video noviji QC, „odglavi” i pomeri sistem napred — što obezbeđuje živost. Sam čin zaključavanja dešava se u fazi COMMIT i predstavlja bezbednosno najosetljiviji korak celog protokola.

3 Faze algoritma

3.1 Bazični HotStuff

U bazičnoj varijanti svaki pogled prolazi kroz četiri faze, strogo istim redosledom:

PREPARE $\rightarrow$ PRE-COMMIT $\rightarrow$ COMMIT $\rightarrow$ DECIDE

PREPARE.

Lider prikuplja NEW-VIEW poruke od kvoruma replika, iz njih bira najviši viđeni QC (**highQC**), pravi novi blok koji ga proširuje i emituje ga svima. Replika glasa za blok samo ako on proširuje blok iz **highQC** i ako zadovoljava **safeNode**.

PRE-COMMIT.

Kada lider skupi kvorum PREPARE glasova, formira `prepareQC` i emituje ga. Prijem `prepareQC` kod replike znači da je blok dobio kvorum u prvoj fazi.

COMMIT.

Nakon kvoruma PRE-COMMIT glasova nastaje `commitQC`. U ovoj fazi replika se *zaključava* na blok (postavlja `lockedQC`). To je korak koji čini da dva konfliktna izvršenja postanu nemoguća.

DECIDE.

Poslednji kvorum glasova daje konačni sertifikat. Replika tada *izvršava* blok — primenjuje njegovu komandu na mašinu stanja (bankarski registar), odgovara klijentu i prelazi u naredni pogled.

Pošto se blok izvršava tek nakon što je prošao kroz sve četiri faze, i to na osnovu tri uzastopna kvoruma, nijedan pojedinačni čvor (pa ni vizantijski lider) ne može da nametne blok bez saglasnosti većine ispravnih replika.

3.2 Promena lidera (view change)

Lider view-a se bira jednostavnim pravilom $\text{leader}(v) = v \bmod n$ (round-robin rotacija). Ako se u nekom pogledu ne donese odluka u zadatom roku (npr. jer je lider otkazao ili ćuti), aktivira se tajmer i replika prelazi na naredni view, šaljući `NEW-VIEW` poruku novom lideru zajedno sa svojim `prepareQC`. Rok se pri svakom neuspehu udvostručuje (eksponencijalni *back-off*), tako da se, kada mreža postane sinhrona, ispravnom lideru sigurno pruži dovoljno vremena da donese odluku.

Bitno je istaći da *ne postoji poseban potprotokol* za promenu lidera: regularna PREPARE faza novog lidera (skupljanje `NEW-VIEW` poruka i izbor najvišeg QC) upravo *jeste* promena pogleda. To je ono što HotStuff-u daje linearnu promenu lidera.

3.3 Ulančani (chained) HotStuff

Bazna varijanta troši četiri runde na jednu komandu. Ulančana varijanta koristi iste poruke, ali ih ukršta kroz pipeline: postoji samo jedna (`GENERIC`) faza po pogledu, a glasovi jednog pogleda služe kao opravdanje predloga u sledećem. Isti QC istovremeno igra ulogu PREPARE za svoj blok, PRE-COMMIT za blok roditelja i COMMIT za blok prethodnika roditelja. Blok se izvršava kada se nađe na čelu *tri uzastopna* pogleda (three-chain):

jedan lanac	osveži <code>genericQC</code>
dva lanca	zaključaj blok
tri lanca	izvrši blok

Zahvaljujući pipeline-u, ulančana varijanta izvršava znatno više komandi u istom vremenu (u priloženom demou približno četiri puta više), uz identične bezbednosne garancije. Sve ostalo — tipovi, kriptografija, stablo, mreža i bankarski registar — ostaje nepromenjeno, razlikuje se samo logika replike.

4 Arhitektura simulatora

Simulator je napisan u Pythonu (verzija 3.11+) i organizovan je u nekoliko malih, jasno razdvojenih modula. Namerno su izbegnute složene apstrakcije i spoljni okviri, da bi kod što neposrednije pratio pseudokod iz rada; fazni rukovaoci nose imena kao u radu (`on_prepare`, `on_pre_commit`, itd.), a stanje replike koristi originalne nazive (`view`, `locked_qc`, `prepare_qc`).

4.1 Organizacija koda

```
hotstuff/          # jezgro konsenzusa (nezavisno od primene)
  types.py         # Node, QC, Msg + kanonsko hesiranje
  crypto.py        # Ed25519 potpisi, izgradnja/provera QC
  tree.py          # blok stablo, extends() / conflicts()
  network.py       # simulirana mreza: send / broadcast, kasnjenje / gubitak
  pacemaker.py     # izbor lidera, tajmeri, back-off
  replica.py       # BAZICNI HotStuff (Algoritam 2)
  chained_replica.py # ULANCANI HotStuff (Algoritam 3)
  client.py        # slanje komandi, prihvatanje na f+1 istih odgovora
  log.py, eventlog.py # strukturisan dnevnik dogadjaja
bank/
  ledger.py        # racuni + deterministicki apply(TRANSFER) -- masina stanja
byzantine.py       # vizantijski lideri: equivocate, censor
demos/            # pet scenarija + zajednicki harness
tests/            # pytest: krypto, stablo, safeNode, konsenzus, ...
gui/              # web prikaz snimljenih izvršavanja
```

4.2 Konkurentno izvršavanje i komunikacija

U skladu sa zahtevom da čvorovi rade konkurentno, a ne kao jedna sekvencijalna petlja, svaka od četiri replike je nezavisan `asyncio` zadatak. Zadatak u petlji čita svoj „sandučić” poruka i granja se prema tipu poruke. Komunikacija ide isključivo porukama preko simulirane mreže: svaka replika ima svoj `asyncio.Queue` kao ulazni red, a magistrala podržava usmereno slanje (`send`), `broadcast` svima, kao i podesivo kašnjenje, treperenje (jitter) i nasumično gubljenje poruka. Poruke se prenose kao objekti po referenci — nema soketa ni serijalizacije, jer je fokus na samom protokolu, a ne na mrežnoj infrastrukturi.

4.3 Mašina stanja: bankarski registar

Modul `bank/ledger.py` sadrži mašinu stanja nad kojom se protokol demonstrira. Funkcija `apply` je čista i deterministička: za dati transfer proverava da li je iznos ispravan, da li je *nonce* već iskorišćen (zaštita od ponavljanja) i da li pošiljalac ima dovoljno sredstava, pa vraća ishod OK ili odgovarajući REJECT. Bitno je da se i odbijena komanda ipak nalazi u dnevniku na svom mestu — ona samo „ne prolazi” pri primeni. Time se direktno ilustruje glavna ideja projekta: konsenzus uređuje, mašina stanja sudi.

4.4 Simulacija grešaka

Zahtev za prikazom grešaka pokriven je na više načina. Otkaz čvora modeluje se tako što se replika zaustavi, a njeno sandučić više ne čita; mrežna particija se postiže potpunim odsecanjem čvora sa magistrale. Kašnjenje i gubitak poruka su parametri mreže. Vizantijsko ponašanje je izdvojeno u dve male potklase ispravne replike (jedino nasleđivanje u celom kodu):

- **Equivocating lider** — kao lider šalje *dva konfliktna* bloka različitim polovinama klastera. Nijedna polovina ne može da skupi kvorum od tri glasa, pa nijedan QC ne nastaje, pogled istekne i preuzima ga ispravan lider. Ovo je „živi dokaz” tvrđenja da u jednom pogledu ne mogu postojati dva konfliktna QC.
- **Censoring lider** — kao lider prećutno odbija da predloži transfere jedne određene banke. Ti transferi ostaju u baferu ispravnih replika i budu predloženi čim rotacija dovede ispravnog lidera, pa se izvrše nakon svega nekoliko pogleda.

Nakon svakog izvršavanja pokreće se i provera bezbednosnih invarijanti: da nijedne dve ispravne replike nisu izvršile konfliktne grane i da po pogledu i tipu postoji najviše jedan QC.

4.5 Prikaz izvršavanja, demo i testovi

Rad algoritma se prati kroz strukturisan dnevnik: svaka faza ispisuje jednu čitljivu liniju (slanje/prijem poruka, promene stanja, trenutak odluke), pa se tok izvršavanja čita gotovo kao priča. Isti tok događaja koristi i opcionalna web „komandna soba” koja reprodukuje snimljena izvršavanja korak po korak (magistralu poruka, stablo blokova, stanje replika i salda računa).

Priloženo je pet demoa: (1) srećan slučaj sa fiksnim liderom; (2) otkaz i kasniji restart lidera uz sustizanje (catch-up); (3) dvostruka potrošnja rešena uređenjem; (4) dva vizantijska lidera; (5) ulančani HotStuff sa poređenjem propusnosti. Svi parametri (kašnjenje/gubitak u mreži, rok pogleda, koji je čvor vizantijski, fiksni ili rotirajući lider, seed) zadaju se kao obični argumenti konstruktora klase `Cluster`, pa se scenariji lako menjaju. Uz demoe je priložen i skup automatskih testova (pytest) koji proverava kriptografiju, stablo, predikat `safeNode`, kompletnu saglasnost dnevnika, promenu pogleda, registar i ulančanu varijantu.

5 Zaključak

U radu je prikazana implementacija simulatora HotStuff protokola i njegova primena na problem međubankarskog poravnanja. Realizovane su obe varijante iz originalnog rada — bazična i ulančana — uz konkurentno izvršavanje čvorova, komunikaciju porukama, kriptografske kvorum sertifikate, rotaciju lidera i promenu view-a, kao i simulaciju otkaza, kašnjenja i vizantijskog ponašanja. Kroz demonstracione scenarije pokazano je da protokol održava jedinstven, potpuno uređen dnevnik transakcija čak i u prisustvu zlonamernog lidera, čime se rešava problem dvostruke potrošnje bez oslanjanja na centralnog posrednika.

Napravljena pojednostavljena (fiksne vrednosti $n = 4$, $f = 1$; eksplicitni potpisi umesto *threshold* sheme; mreža unutar jednog procesa) ne menjaju logiku protokola i jasno su dokumentovana. Kao mogući pravci daljeg rada izdvajaju se trajno čuvanje stanja i oporavak nakon pada, dinamičko članstvo, upotreba pravih *threshold*/BLS potpisa, stvarna mrežna komunikacija i optimizacija performansi.